

Notification-Kit Paketini Projenize Entegre Etme

Bu sunum, notification-kit paketini kendi Django projenize nasıl entegre edeceğinizi adım adım anlatır. Paket sayesinde e-posta, SMS ve push notification gönderimini kolayca yönetebilir, kullanıcı tercihlerini takip edebilir ve asenkron bildirim sistemleri kurabilirsiniz.



Paketi Yükleme ve Yapılandırma

Kurulum Seçenekleri

Paketi üç farklı yöntemle yükleyebilirsiniz: PyPI'dan (pip install notification-kit), GitHub'dan (pip install git+https://github.com/username/notification-kit.git) veya yerel dosyadan (pip install /path/to/notification-kit/).

Kurulumdan sonra settings.py dosyanıza 'notification_kit' uygulamasını INSTALLED_APPS listesine ekleyin. Rest framework da gereklidir.



Paket Ayarlarını Yapılandırma

settings.py dosyanızda NOTIFICATION_KIT sözlüğünü oluşturarak temel ayarları yapılandırın. Varsayılan kanalı (DEFAULT_CHANNEL), maksimum tekrar deneme sayısını (MAX_RETRIES) ve her kanal için backend ayarlarını belirleyin.

E-posta Backend

FROM_EMAIL adresini belirleyin

SMS Backend

Twilio için ACCOUNT_SID, AUTH_TOKEN ve FROM_NUMBER gerekli

Push Backend

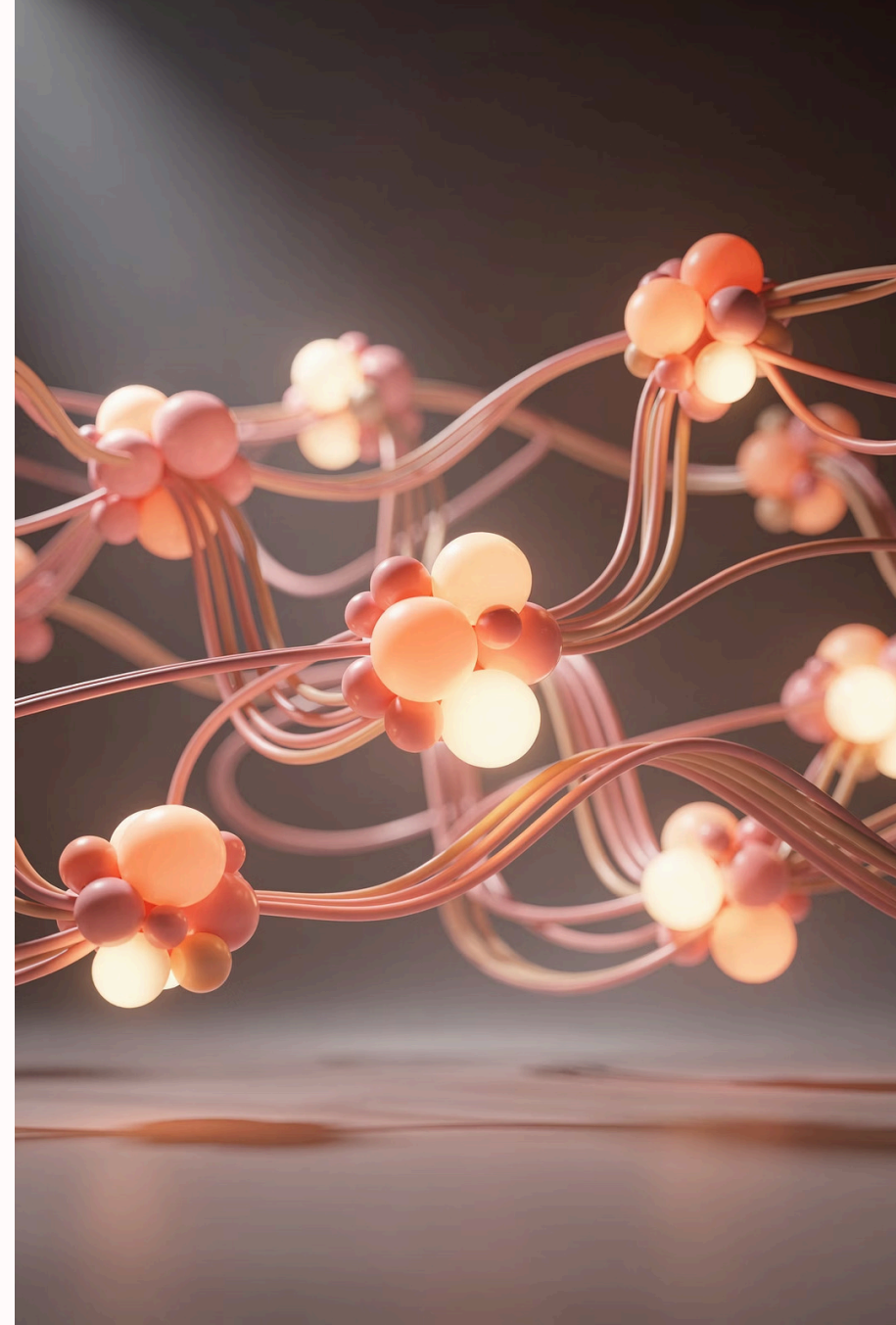
Firebase için credentials dosya yolunu tanımlayın

MODELLER

Neden Kendi Modelinizi Oluşturmalısınız?

Paket abstract modeller sağlar ve siz bunları miras alarak kendi modellerinizi oluşturursunuz. Bu yaklaşım size önemli avantajlar sunar: Kendi kullanıcı modelinizi kullanabilir, ekstra alanlar ekleyebilir, farklı ilişkiler tanımlayabilir ve projenize özel ihtiyaçlara göre özelleştirebilirsiniz.

İlk adım olarak yeni bir uygulama oluşturun: `python manage.py startapp notifications`



Model Tanımlamaları

Üç Temel Model

- Notification: Bildirim kayıtları
- NotificationPreference: Kullanıcı tercihleri
- NotificationTemplate: Bildirim şablonları

notifications/models.py dosyanızda AbstractNotification, AbstractNotificationPreference ve AbstractNotificationTemplate sınıflarını miras alın. Her model için zorunlu user ilişkisini tanımlayın.

Notification modeline priority (öncelik) ve related_order (ilgili sipariş) gibi ekstra alanlar ekleyebilirsiniz. NotificationTemplate modeline category (kategori) alanı ekleyerek şablonları gruplandırabilirsiniz.

Migrasyon ve Admin Paneli

01

Migrasyonları Oluşturun

python manage.py makemigrations
notifications ve python manage.py migrate
komutlarını çalıştırın

03

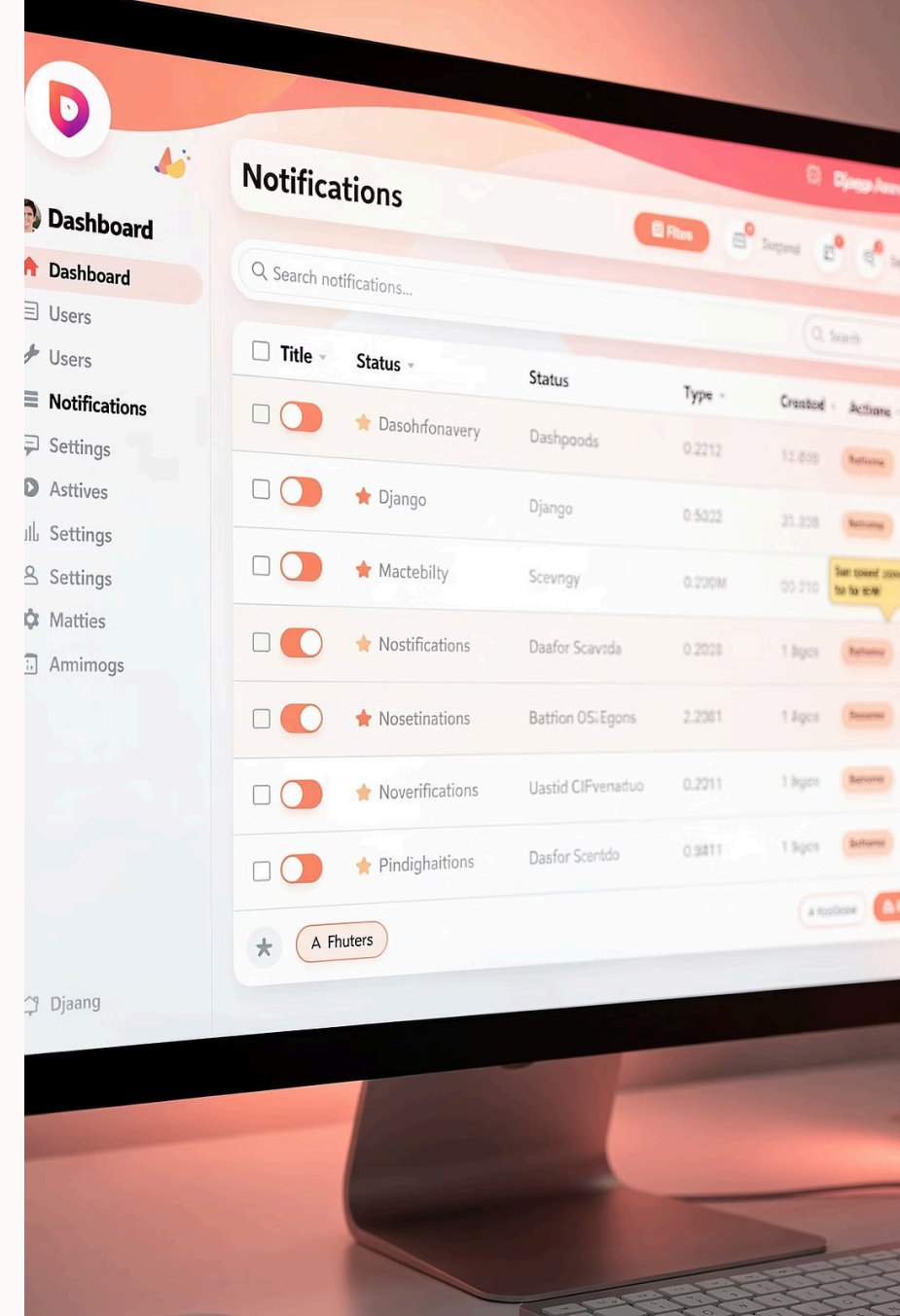
Görünümleri Özelleştirin

list_display, list_filter, search_fields ve fieldsets ile admin panelini yapılandırın

02

Admin Sınıflarını Tanımlayın

notifications/admin.py dosyasında her model
için admin sınıfı oluşturun



NotificationService ile Temel Kullanım



Tek Bildirim Gönderme

NotificationService sınıfını import edin ve send() metodunu kullanarak tek bir kullanıcıya bildirim gönderin. Parametreler: user, channel, title, body ve priority.

Toplu Bildirim Gönderme

send_bulk() metodu ile birden fazla kullanıcıya aynı anda bildirim gönderebilirsiniz. Users parametresine QuerySet geçirin.

Template ile Dinamik İçerik

`render_template_string` fonksiyonu ile dinamik içerik oluşturabilirsiniz. Context sözlüğünde kullanıcı adı, sipariş numarası, toplam tutar gibi değişkenleri tanımlayın ve şablon içinde `{{ değişken_adı }}` formatıyla kullanın.

Context Oluşturma

Dinamik verileri içeren bir sözlük hazırlayın

Şablon Render

Title ve body için `render_template_string` kullanın

Bildirim Gönderimi

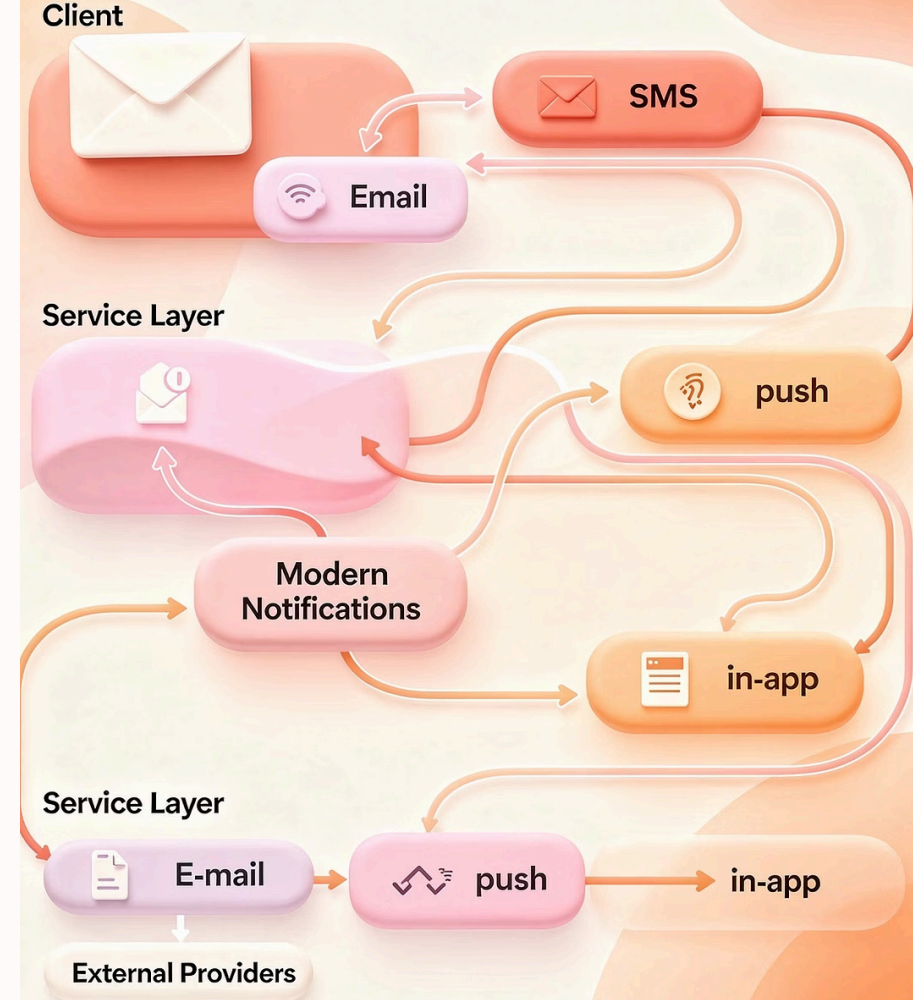
Render edilmiş içeriği `service.send()` ile gönderin

Özel Servis Sınıfı Oluşturma

NotificationService sınıfını miras alarak projenize özel metodlar ekleyebilirsiniz. MyNotificationService sınıfında send_welcome_email(), send_order_confirmation() ve send_payment_reminder() gibi özel metodlar tanımlayın.

Her metod belirli bir iş senaryosu için optimize edilmiş bildirim gönderir. Örneğin send_payment_reminder() hem e-posta hem SMS gönderebilir ve kullanıcı tercihlerini kontrol eder.

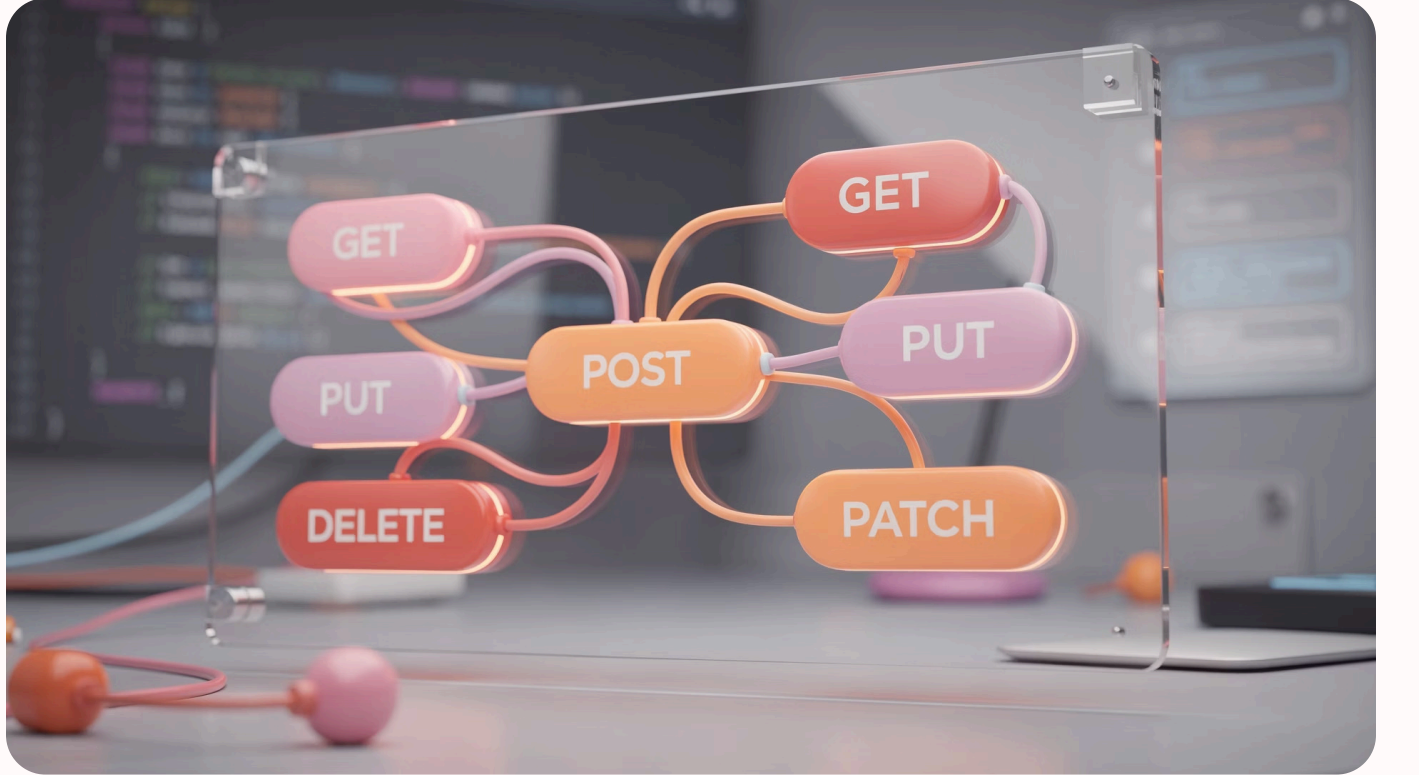
Custom Service Class Architecture



API Endpoint'leri ve Kullanımı

Mevcut Endpoint'ler

- GET /api/notifications/ - Bildirimleri listele
- GET /api/notifications/{id}/ - Bildirim detayı
- POST /api/notifications/{id}/read/ - Okundu işaretle
- GET /api/notifications/unread_count/ - Okunmamış sayısı
- GET /api/preferences/ - Tercihleri listele
- PATCH /api/preferences/{id}/ - Tercihi güncelle



Ana urls.py dosyanıza `path('api/notifications/', include('notification_kit.api.urls'))` satırını ekleyerek API'ları aktif edin.

Frontend Entegrasyonu

JavaScript ile API'leri kullanarak bildirimleri yönetebilirsiniz. Async/await yapısı ile `getNotifications()`, `getUnreadCount()`, `markAsRead()` ve `updatePreference()` fonksiyonlarını oluřturun.



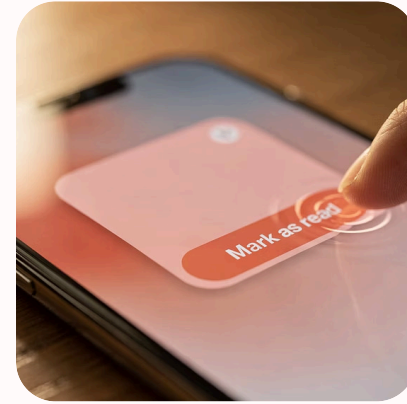
Bildirimleri ekme

`fetch()` ile `/api/notifications/ endpoint'`ini aęırın



Okunmamıř Sayısı

`unread_count` deęerini alıp UI'da gsterin



Okundu İřaretleme

POST isteęi ile bildirimi okundu olarak iřaretleyin



ASENKRON İŞLEMLER

Celery ile Asenkron Gönderim

E-posta veya SMS göndermek yavaş işlemlerdir (2-5 saniye). Kullanıcıyı bekletmemek için Celery ile arka planda çalıştırmalısınız. Celery ve Redis'i yükleyin: `pip install celery redis`

settings.py'de `CELERY_BROKER_URL` ve `CELERY_RESULT_BACKEND` ayarlarını yapılandırın.

`myproject/celery.py` dosyasında Celery uygulamasını oluşturun ve `autodiscover_tasks()` ile task'ları otomatik keşfedin.

Celery Task Tanımlama

Task Örnekleri

notifications/tasks.py dosyasında @shared_task dekoratörü ile task'lar tanımlayın. send_notification_task() tek bildirim gönderir ve hata durumunda 60 saniye sonra tekrar dener (max_retries=3).

send_bulk_notifications_task() toplu bildirim gönderir.

send_scheduled_notifications() zamanlanmış bildirimleri kontrol eder ve Celery Beat ile periyodik çalıştırılır.

Kullanım

View'da senkron yerine asenkron gönderim yapın:

```
send_notification_task.delay(notification.id)
```

Bu sayede kullanıcı beklemez ve işlem arka planda tamamlanır.

Paketin Sinyallerini Dinleme

notification_created, notification_sent ve notification_failed sinyallerini dinleyerek özel işlemler yapabilirsiniz. notifications/signals.py dosyasında @receiver dekoratörü ile sinyal fonksiyonları tanımlayın.



notification_created

Bildirim oluşturulduğunda analytics'e gönder



notification_sent

Başarı metriğini artır

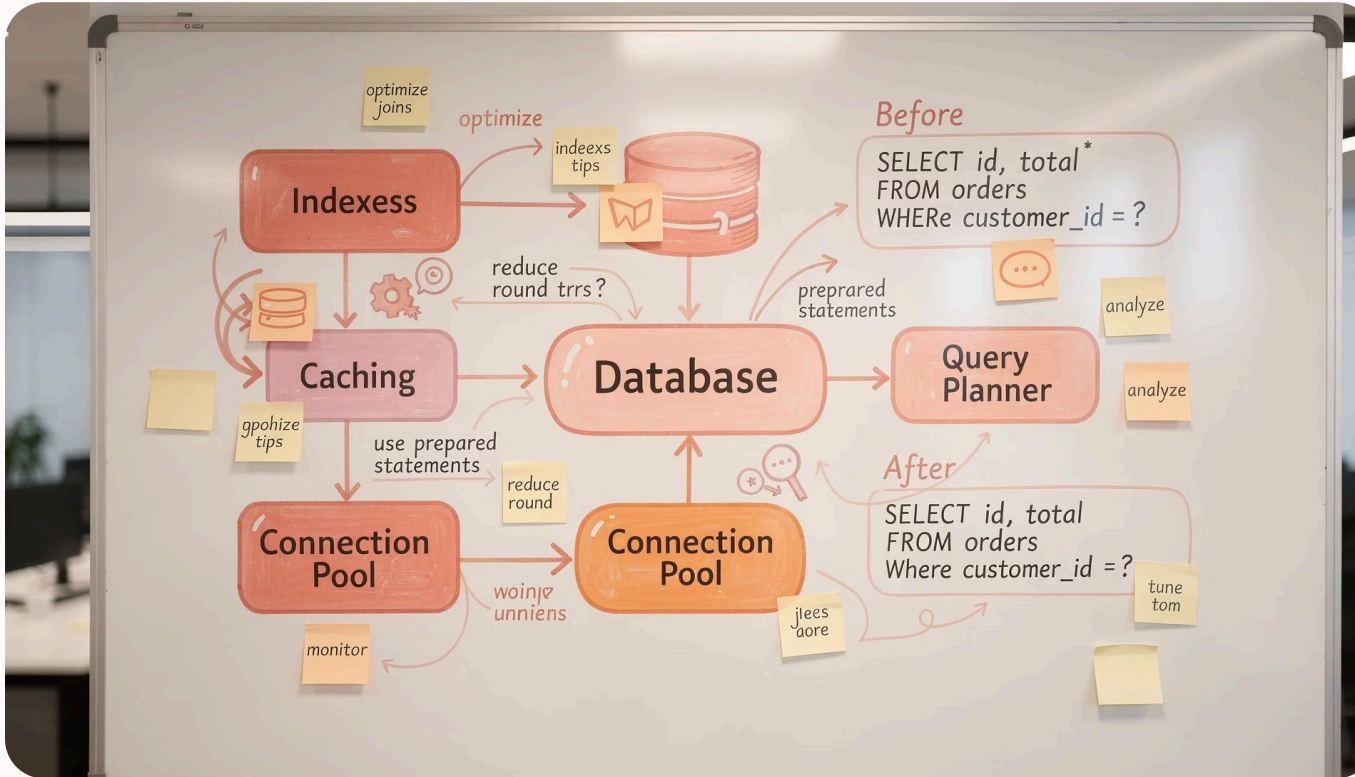


notification_failed

Hata logla ve acil durumlarda ekibi uyar

notifications/apps.py dosyasının ready() metodunda sinyalleri import ederek bağlantıları aktif edin.

Veritabanı Performansı ve Toplu İşlemler



N+1 Sorgu Problemini Önleyin

`select_related()` kullanarak ilişkili nesneleri tek sorguda çekin. Yanlış: Her iterasyonda ayrı sorgu. Doğru: `Notification.objects.select_related('user').all()`

Toplu İşlemlerde `bulk_create()`

Çok sayıda kayıt eklerken her biri için ayrı `INSERT` yerine `bulk_create()` ile tek sorguda ekleyin. Bu yöntem performansı önemli ölçüde artırır.

Rate Limiting ve Kullanıcı Tercihleri



Rate Limiting

Harici servislerin hız limitleri vardır. NotificationRateLimitError yakalayın, retry_after süresini bekleyin ve tekrar deneyin.



Kullanıcı Tercihlerine Saygı

Pazarlama emaili göndermeden önce NotificationPreference kontrolü yapın. Kullanıcı email istemiyorsa göndermeyin, sessiz saatlerde ise zamanla.



Hata Yönetimi

NotificationTemplateError, NotificationRateLimitError ve NotificationSendError'ları yakalayın. Hataları logla ve gerekirse Celery task'a atarak tekrar deneyin.

FAYDALAR

Sağlanan Faydalar

12-19

Gün Tasarruf

Sıfırdan sistem yazmak 12-19 gün sürerken paket ile 1-2 günde tamamlanır

157

Test Sayısı

Paket 157 test ile gelir ve tüm senaryoları kapsar

100%

Standart Yapı

Kanıtlanmış tasarım kalıpları kullanılır

Time Savings Comparison



Geniřletilebilirlik ve Bakım Kolaylıęı

Kendi Backend'inizi Ekleyin

BaseBackend sınıfını miras alarak özel backend'ler oluşturabilirsiniz. Örneęin SlackBackend ile Slack entegrasyonu yapabilir, send() metodunu override ederek Slack API'yi çağırabilirsiniz.

Bu esneklik sayede herhangi bir bildirim kanalını sisteme entegre edebilirsiniz.

Bakım Avantajları

- Tek bir yerde güncelleme
- Versiyon yönetimi
- Kapsamlı dokümantasyon
- Topluluk desteęi (açık kaynak ise)
- Sürekli iyileřtirme

ÖRNEK SENARYO

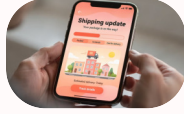
E-Ticaret Uygulaması Entegrasyonu

Bir e-ticaret sitesi için ECommerceNotificationService sınıfı oluşturun. Bu sınıf NotificationService'i miras alır ve özel metodlar içerir.



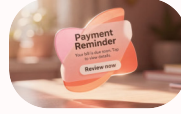
Sipariş Onayı

notify_order_placed()
metodu sipariş
alındığında email
gönderir



Kargo Bildirimi

notify_order_shipped(
) hem email hem SMS
ile kargo bilgisi
gönderir



Ödeme Hatırlatması

notify_payment_due()
vade tarihine göre
öncelikli bildirim
gönderir



Kampanya Bildirimi

notify_campaign()
toplu bildirim
gönderir ve tercihleri
filtreler

View'larda Kullanım ve Sonuç

OrderViewSet Örneği

ECommerceNotificationService'i import edin ve view'larda kullanın. perform_create() metodunda sipariş oluşturulduğunda notify_order_placed() çağırın.

ship() action'ında sipariş kargoya verildiğinde notify_order_shipped() ile bildirim gönderin. Bu sayede tüm bildirim mantığı servis katmanında kalır ve view'lar temiz olur.

Sonuç

notification-kit paketi ile profesyonel bir bildirim sistemi kurabilir, zaman tasarrufu sağlayabilir ve test edilmiş, genişletilebilir bir yapı elde edebilirsiniz.

Paket sayesinde e-posta, SMS ve push notification yönetimi kolaylaşır, kullanıcı tercihleri takip edilir ve asenkron işlemler sorunsuz çalışır.